City Care

Flujo ciudadano móvil y seguimiento administrativo desde la PC

MÓVIL	BACKEND	PANEL WEB
Ciudadano autenticado	AdonisJS + MySQL + MinIO	Administrador y dependencia

Objetivo	Comprobar el recorrido completo: iniciar sesión desde Flutter, enviar un reporte con fotografía y GPS, procesarlo en el backend y consultarlo desde el panel administrativo.

Estado real verificado en MySQL

Elemento	Estado actual
Cuenta ciudadana	Creada y activa
Cuenta administradora global	Creada y activa
Categorías	9 activas; todavía sin dependencia asignada
Equipos de trabajo	0 registrados
Reportes	0 registrados

Conclusión rápida	El endpoint administrativo ya existe y el aislamiento por dependencia ya está implementado. Falta preparar los datos operativos: dependencias reales, relación categoría-dependencia, administradores de dependencia y equipos.

1. Cuenta para probar la aplicación móvil

Correo	**ciudadano@citycare.com**

Contraseña	**CiudadanoCityCare123!**

Uso	Credencial exclusivamente local y de desarrollo. Debe cambiarse o eliminarse antes de publicar el sistema.

Dirección del backend

Origen de Flutter	Base URL
Android Emulator	http://10.0.2.2:3333
Flutter web en la misma PC	http://127.0.0.1:3333
Celular físico en la misma red	http://IP_LOCAL_DE_LA_PC:3333

Recorrido esperado

- **1.** Flutter envía correo y contraseña a **POST /api/v1/auth/login**.
- **2.** El backend valida la tabla credentials y devuelve un token Bearer.
- **3.** Flutter guarda el token con flutter_secure_storage.
- **4.** Consulta **GET /api/v1/report-categories** para llenar la lista.
- **5.** Captura fotografía, latitud, longitud, precisión, dirección y descripción.
- **6.** Envía el reporte autenticado como multipart/form-data.
- **7.** Recibe HTTP 201, folio y URL firmada de la fotografía.

```
Authorization: Bearer TOKEN_DEL_LOGIN
```

2. Envío exitoso del reporte

Endpoint ciudadano: **POST /api/v1/reports**. El usuario del reporte no se envía: se obtiene del token autenticado.

Campo	Tipo	Regla
categoria	string	Nombre exacto de una categoría activa
foto	archivo	JPG, JPEG o PNG; máximo 10 MB
latitud	number	Entre -90 y 90; 0,0 no permitido
longitud	number	Entre -180 y 180; 0,0 no permitido
precisionUbicacion	number opcional	Precisión reportada por GPS
origenUbicacion	string	gps o manual
direccion	string opcional	Dirección visible para el administrador
descripcion	string opcional	Obligatoria cuando la categoría es Otro

Procesamiento interno

- La imagen se valida y se prepara temporalmente.
- La imagen se almacena en el bucket privado **city-care** de MinIO.
- MySQL guarda el texto, coordenadas, ciudadano, categoría, dependencia y referencia del objeto.
- Se crea el estado inicial **pending** y su registro de historial.
- El servidor genera un folio como **CC-AÑO-000001**.
- La respuesta incluye una URL firmada temporal; Flutter no debe conservarla como permanente.

Respuesta correcta	HTTP 201. El reporte queda persistido y puede consultarse inmediatamente desde Mis reportes y desde el panel administrativo que tenga alcance.

Errores principales

HTTP	Significado
401	Token ausente, vencido o revocado
422	Campos, categoría, GPS o fotografía inválidos
503	MinIO no disponible

3. Acceso administrativo desde la PC

Correo	**admin@citycare.com**

Contraseña local	**admincitycare123**

Seguridad	Esta es una credencial local utilizada durante el desarrollo. No debe publicarse ni reutilizarse en producción.

Inicio de sesión

- Iniciar backend en el puerto 3333 y frontend Nuxt normalmente en el puerto 3000.
- Abrir **http://localhost:3000** e iniciar sesión.
- El administrador global actualmente es enviado a **/admin/users**.
- Para revisar reportes puede abrir **/dependencia/alumbrado-publico**; la ruta acepta admin y dependency_admin.

Funciones ya conectadas en la vista

- Carga de reportes y equipos reales.
- Búsqueda por folio, descripción o ubicación; filtro por estado y paginación.
- Detalle con fotografía de MinIO, ciudadano, coordenadas, descripción y equipo.
- Asignación o reasignación de equipo.
- Cambio normal de Pendiente a En proceso y de En proceso a Resuelto.
- Descarga CSV de los resultados visibles.

Pendiente visual	Agregar en el panel del administrador global un enlace directo a Reportes. Por ahora puede entrar escribiendo la ruta indicada.

4. Endpoint administrativo y aislamiento

Respuesta	**No falta el endpoint.** Ya existe GET /api/v1/admin/reports y está protegido por autenticación y el permiso reports.list.

Endpoint	Administrador global	Administrador de dependencia
GET /api/v1/admin/reports	Todos los reportes	Solo reportes con su dependency_id
GET /api/v1/admin/reports/:id	Cualquier reporte	Detalle únicamente de su dependencia
PATCH /api/v1/admin/reports/:id/status	Puede actualizar	Solo dentro de su dependencia
GET /api/v1/admin/teams	Todos o filtrados	Solo equipos de su dependencia
POST /api/v1/admin/reports/:id/assignments	Puede asignar	Reporte y equipo deben ser de su dependencia

Validaciones existentes

- El middleware comprueba permisos según rol antes de entrar al controlador.
- El rol **user** no tiene permisos administrativos y recibe HTTP 403.
- El listado fuerza dependency_id del usuario cuando no es administrador global.
- El detalle y cambio de estado rechazan reportes de otra dependencia con HTTP 403.
- El listado de equipos fuerza la dependencia del administrador autenticado.
- La asignación comprueba que reporte y equipo pertenezcan a la misma dependencia.
- Las pruebas funcionales ya comprueban el aislamiento y los rechazos entre dependencias.

Confirmación	No falta crear estas validaciones principales. Lo que falta es completar los datos reales y ejecutar una prueba integral con las cuentas definitivas de cada dependencia.

5. Pendiente crítico en la base de datos

La consulta de MySQL muestra únicamente las dependencias **Administracion** y **Ciudadania**. Las nueve categorías tienen dependency_id en NULL, no existen equipos y todavía no hay reportes.

Impacto	Un reporte creado hoy queda sin dependencia. El administrador global sí puede verlo, pero un administrador de Alumbrado Público no lo verá porque el backend filtra por dependency_id.

Preparación necesaria

- Crear las dependencias operativas: Alumbrado Público, Obras Públicas, Agua, Aseo, Movilidad, Parques u otra estructura definida por el municipio.
- Relacionar cada categoría con la dependencia responsable.
- Crear por lo menos una cuenta con rol dependency_admin por dependencia.
- Crear líderes, integrantes y equipos para poder asignar incidencias.
- Mantener Otro bajo una dependencia de recepción o definir un flujo de clasificación administrativa.

Ejemplo mínimo para Alumbrado Público

```
INSERT INTO dependencies (name, description, is_active, created_at, updated_at)
SELECT 'Alumbrado Publico', 'Atencion de luminarias y red de alumbrado', 1, NOW(), NOW()
WHERE NOT EXISTS (
  SELECT 1 FROM dependencies WHERE name = 'Alumbrado Publico'
);

UPDATE report_categories
SET dependency_id = (
  SELECT id FROM dependencies WHERE name = 'Alumbrado Publico' LIMIT 1
)
WHERE code = 'public_lighting';
```

Recomendación	Crear la cuenta del administrador y los equipos mediante los endpoints administrativos para conservar validaciones, hash de contraseña y restricciones de rol. No insertar contraseñas directamente con SQL.

6. Prueba integral de aceptación

Paso	Acción	Resultado esperado
1	Preparar Alumbrado Público y relacionar la categoría	dependency_id válido
2	Crear admin de dependencia y equipo activo	Cuenta, líder e integrantes disponibles
3	Iniciar backend, MinIO y frontend	Puertos 3333, 9000 y 3000 disponibles
4	Login móvil con la cuenta ciudadana	HTTP 200 y token guardado
5	Enviar Alumbrado público con foto y GPS	HTTP 201 y folio generado
6	Consultar Mis reportes	El ciudadano ve únicamente su reporte
7	Login de admin de Alumbrado Público	Acceso al panel de dependencia
8	Abrir la bandeja	Aparece el reporte y su fotografía
9	Asignar un equipo	Asignación activa de la misma dependencia
10	Cambiar a En proceso y Resuelto	Historial actualizado
11	Login de otra dependencia	No ve ni puede modificar el reporte

Resultado para considerar el flujo terminado

- La app móvil puede iniciar sesión, obtener categorías y enviar un reporte con evidencia.
- MySQL conserva el reporte y MinIO conserva la fotografía.
- El ciudadano consulta solamente sus propios reportes.
- El administrador global consulta todos los reportes.
- Cada administrador de dependencia consulta y administra únicamente los suyos.
- Los equipos solo pueden recibir incidencias de su misma dependencia.

Siguiente trabajo recomendado	Configurar primero Alumbrado Público en MySQL, crear su administrador y su primer equipo. Después ejecutar este checklist completo antes de preparar las demás dependencias.